Universidad Autónoma de Ciudad Juárez

Instituto de Ingeniería y Tecnología

Departamento de Ingeniería Eléctrica y Computación



Tarea 15 Encendido y apagado de un foco con aplausos

Sensores e Instrumentación

Ángel Alvídrez 225581:

Fecha: 6 de Mayo 2025

Índice

1. Introducción **2**

2. Definiciones y marco conceptual **3**

2.1. Micrófono KY-038 3

2.2. Amplificador operacional TL081 4

2.3. Filtro pasabanda activo 4

2.4. Comparador con nivel de referencia ajustable 5

2.5. Optoacoplador MOC3022 6

2.6. TRIAC 2N6071 6

3. Desarrollo **7**

3.1. Amplificación con op amp (TL081) 7

3.2. Filtro pasabanda activo (3kHz – 6kHz) 7

3.3. Comparador de umbral 9

3.4. Procesamiento Digital 10

3.5. Etapa de potencia 10

4. Código **11**

5. Resultados **13**

6. Conclusión **15**

1. Introducción

En la presente práctica se diseña e implementa un sistema capaz de encender y apagar un foco de 110 V a través de la detección de aplausos. Para ello, se emplea una cadena de procesamiento de señales que va desde la captación del sonido hasta la conmutación de un triac mediante un Arduino Uno, se espera que el circuito:

- Detecte el sonido de los aplausos y convertirlo en una señal eléctrica procesable.
- Filtre y amplifique únicamente la banda de frecuencias donde predominan los aplausos (3 kHz - 6 kHz).
- Genere un pulso digital ajustable para el Arduino, en función de la intensidad del aplauso.
- Cuente el número de aplausos en un intervalo de 3 segundos para decidir el encendido (2 aplausos) o apagado (4 aplausos) del foco.
- Conmute un triac (2N6071) con un optoacoplador MOC3022 para aislar la señal de control de la línea de 110 VAC.

La ganancia se ajusta con un potenciómetro en la red de realimentación (Rf), permitiendo controlar la amplitud de la señal proveniente del micrófono KY–038. Se almientará el circuito con +5 V provenientes del Arduino y –5 V generados con un regulador L7905 conectado a una pila de 9V.
El filtro fue implementado con un segundo TL081 diseñado para atenuar todas las frecuencias fuera del rango de interés, asegurando que sólo los aplausos generen una señal significativa.
Un tercer TL081 funciona como comparador de ventana simple. El umbral de disparo se ajusta con otro potenciómetro, de modo que la salida se satura y entrega un nivel digital (HIGH) cuando la señal filtrada supera el nivel de referencia.
La salida del comparador se conecta al pin 2 del Arduino, que detecta flancos de subida para incrementar un contador de aplausos.
Se mide un intervalo de 3 segundos (iniciado con el primer aplauso) dentro del cual: 2 aplausos el Arduino enciende el LED de simulación (y envía la señal al MOC3022 para encender el triac).

4 aplausos el Arduino apaga el LED (y deja de disparar el triac). Valores excedentes de aplausos o fin de los 3 segundos reinician el conteo. El Arduino controlará un MOC3022, que optoaisla la señal de control y dispara el triac 2N6071, encargado de conmutar el foco de 110 VAC.

2. Definiciones y marco conceptual

2.1. Micrófono KY-038

Es un módulo sensor de sonido con salida analógica y digital. En este proyecto solo se utiliza la salida analógica, que proporciona una señal proporcional a la intensidad del sonido captado. El sensor consiste en un micrófono de condensador electret y un pequeño amplificador integrado. La señal es débil y requiere amplificación adicional para ser útil en procesamiento de señales.

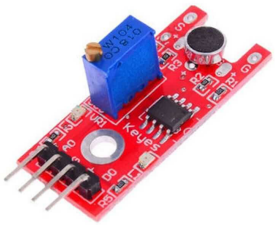


Figura 1: Sensor Ultrasónico HC-SR04

2.2. Amplificador operacional TL081

Es un amplificador operacional de bajo ruido con entrada diferencial y salida simple. Se utilizó este componente para todas las etapas analógicas del circuito:

- Amplificación inicial de la señal del micrófono
- Filtro pasabanda
- Comparador de nivel

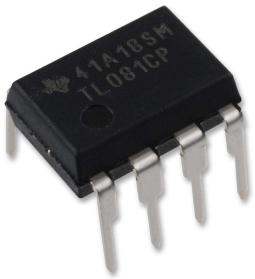


Figura 2: Op Amp TL081

2.3. Filtro pasabanda activo

Los aplausos tienen un componente fuerte de energía entre los 3kHz y 6kHz. Para asegurarnos de que solo estas frecuencias pasen y el sistema no reaccione ante ruidos ambientales u otras voces, se diseñó un filtro pasabanda activo con op amps.
Este filtro atenúa las señales fuera de esa banda, disipando ruido bajo (habla, ambiente) y alto (chispeos, interferencia).

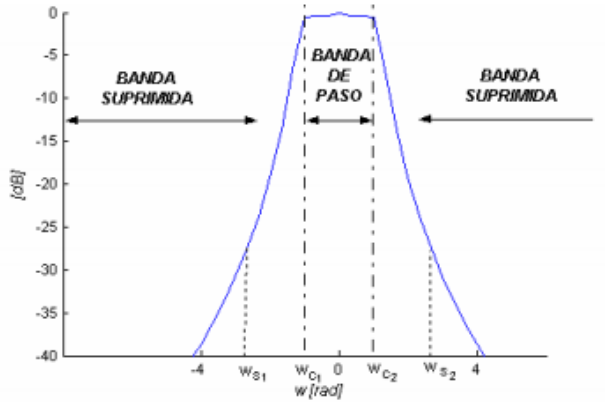


Figura 3: Salida de un filtro pasa banda

2.4. Comparador con nivel de referencia ajustable

La señal filtrada pasa a un comparador construido con otro TL081, la cuál se compara contra un voltaje de referencia ajustable con potenciómetro, cuando la señal supera ese umbral (por un aplauso), el op amp satura y su salida cambia bruscamente a nivel alto (5V), actuando como un detector de aplauso.

La salida es una señal digital que el Arduino puede leer fácilmente.

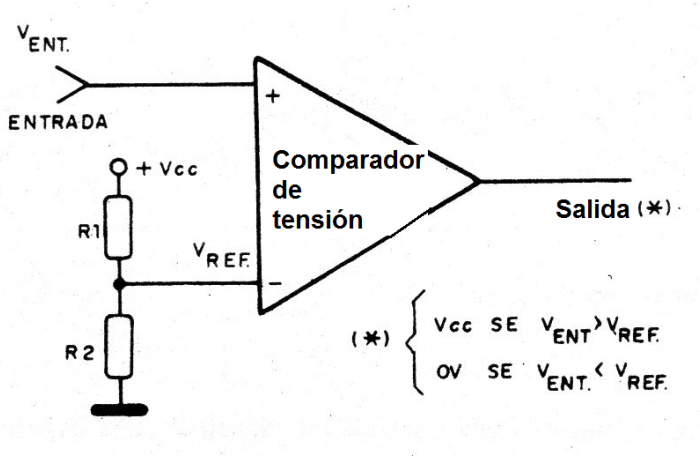


Figura 4: Ejemplo de un comparador de voltaje

2.5. Optoacoplador MOC3022

El MOC3022 es un optoacoplador con salida de TRIAC diseñado para controlar cargas de corriente alterna sin necesidad de conexión directa entre la parte de control y la parte de potencia. Consiste en un diodo LED interno que, al activarse, dispara un TRIAC integrado.

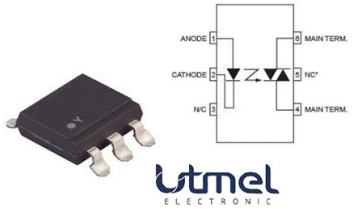


Figura 5: Optoacoplador MOC3022

2.6. TRIAC 2N6071

El 2N6071 es un TRIAC, un dispositivo semiconductor que permite el control de corriente alterna en ambas polaridades de la onda de CA. Funciona como un interruptor controlado por una pequeña señal de disparo en su compuerta

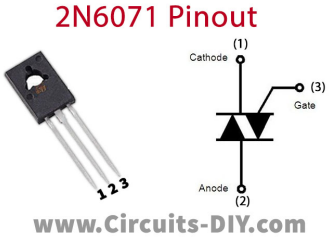


Figura 6: Triac 2N6071

3. Desarrollo

Para el diseño del circuito se dividió todo en bloques funcionales, cada uno con una tarea clara. El primero es la etapa de captación de sonido, que se hizo con un micrófono KY-038. Este módulo tiene dos salidas, una digital y una analógica; para este proyecto se usó la analógica porque entrega una señal proporcional a la intensidad del sonido. Esa señal es de muy baja amplitud, apenas en milivoltios, por lo que se necesitó una etapa de amplificación para poder procesarla bien.

3.1. Amplificación con op amp (TL081)

La amplificación se logró usando un TL081 en configuración no inversora, alimentado con una fuente simétrica de ±5V. Los +5V los tomamos directamente del Arduino, y para los –5V usamos un regulador L7905 con una fuente externa de –9V. En esta etapa también se colocó un potenciómetro de precisión como resistencia variable de realimentación, para ajustar la ganancia del amplificador y así poder controlar qué tan fuerte debía sonar un aplauso para que se detectara.

3.2. Filtro pasabanda activo (3kHz – 6kHz)

Antes de escoger las frecuencias para el filtro, se decidió utilizar el programa Audacity para analizar las frecuencias que hay en los aplausos, por lo que se grabó un aplauso y se aplicó la herramienta de análisis de frecuencia.

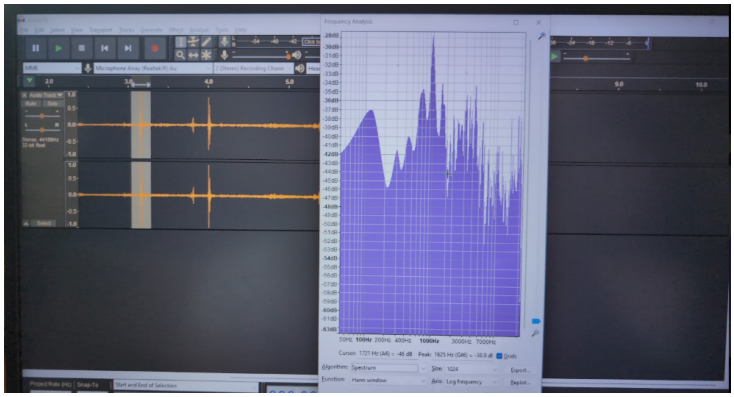


Figura 7: Análisis de una frecuencia en Audacity

Viendo que una gran parte de las frecuencias de los aplausos se aplicó en el mismo programa, un filtro con una curva simulando la que realizará el filtro del circuito.

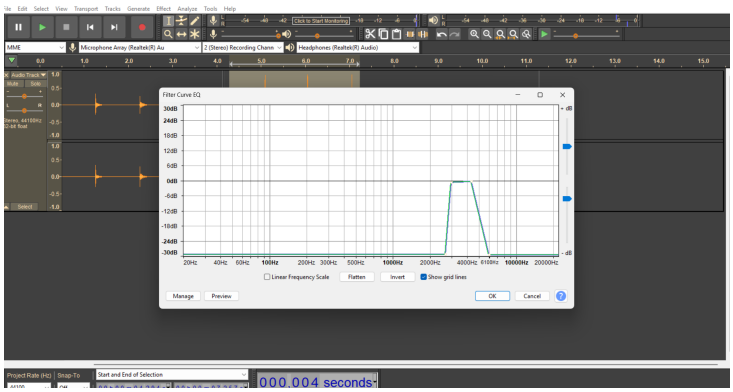


Figura 8: Simulación del filtro en Audacity

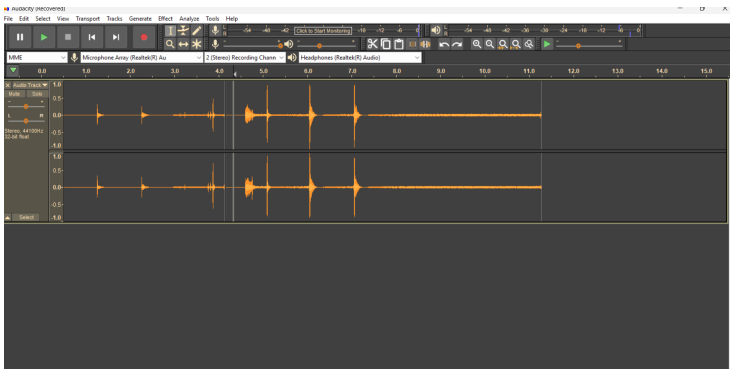


Figura 9: Salida del filtro en Audacity

Una vez aplicado el filtro, se puede observar que se filtran los demás sonidos de fondo del audio y quedan unos picos en el espectrograma del audio, mostrado en la figura 9 demostrando que los aplausos prevalecen después del filtro.

Una vez seleccionadas unas frecuencias para el filtro, se empezó con el diseño de este. El capacitor fue seleccionado de forma práctica, eligiendo un valor de 1nF ya que es un valor fácil de conseguir. A partir de ese valor, se calcularon las resistencias necesarias para centrar el filtro en la frecuencia media de:

$$f_r = \sqrt{f_H \cdot f_L} = \sqrt{6\,\text{kHz} \cdot 3\,\text{kHz}} = 4{,}242\,\text{kHz}$$

La banda pasante se definió como:

$$B = f_H - f_L = 6\,\text{kHz} - 3\,\text{kHz} = 3\,\text{kHz}$$

Y la calidad del filtro como:

$$Q = \frac{f_r}{B} = \frac{4{,}242}{3} = 1{,}414$$

Con estos datos, y usando la configuración de un filtro pasa banda activo con amplificador operacional, se usaron las fórmulas correspondientes para calcular los valores de las resistencias, con la constante del filtro dada por:

$$\frac{1}{RC} = 2\pi B$$

$$R = \frac{1}{2\pi BC} \approx \frac{0{,}1591}{3kHz \times 1nF} \approx 53{,}033k\Omega$$

Para la siguiente resistencia se utilizó la expresión

$$fr = \frac{\sqrt{1 + \frac{R}{R_r}}}{2\sqrt{2} \times \pi \times R \times C}$$

$$R_r = R\left(\left(\frac{f_r \cdot RC}{0{,}1125}\right)^2 - 1\right)^{-1}$$

$$R_r \approx 17{,}684\,\text{k}\Omega$$

3.3. Comparador de umbral

Después del filtro, la señal se envió a una tercera etapa: un comparador de voltaje hecho también con un TL081. Aquí se comparaba la señal del filtro contra un voltaje de referencia ajustable con potenciómetro. Si la señal superaba ese umbral, el comparador saturaba a su voltaje alto (cercano a 5V), generando una salida digital que representaba la detección de un aplauso. Esta salida se conectó directamente a una entrada digital del Arduino

Uno, que se encargó de procesar los eventos Estas 3 etapas anteriores fueron simuladas en Proteus 10para observar su funcionamiento mostrado en la figura 11.

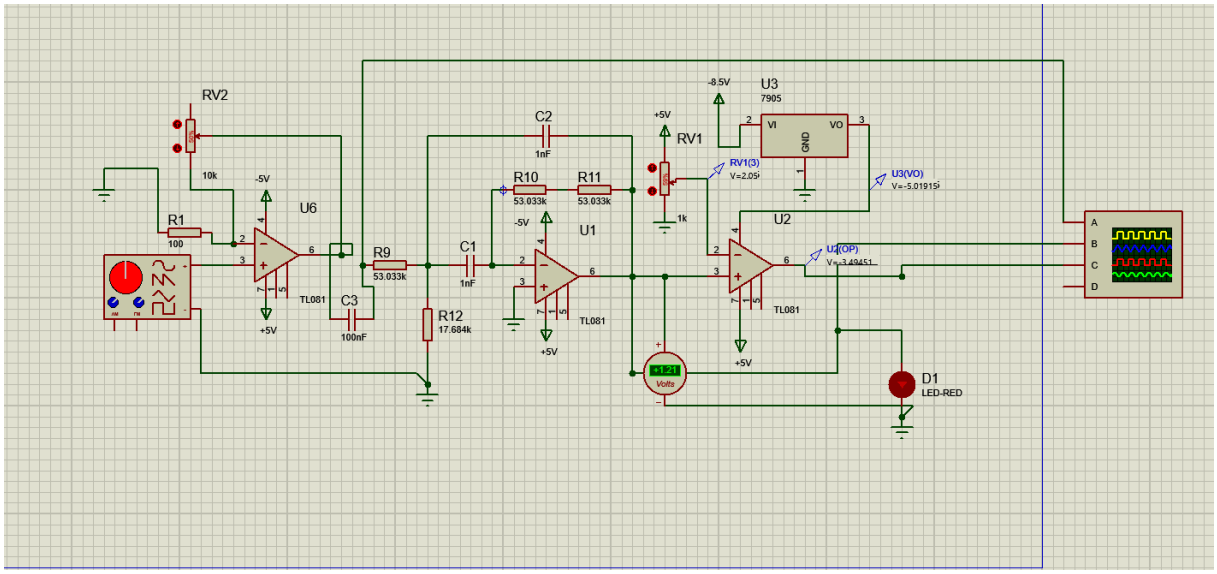


Figura 10: Analisis de una frecuencia en Audacity

3.4. Procesamiento Digital

El Arduino fue programado para detectar los flancos de subida, es decir, cuando la señal pasaba de bajo a alto. Al detectar un aplauso, iniciaba una ventana de 3 segundos para contar los siguientes.

Si dentro de ese tiempo se contaban 2 aplausos, se encendía el foco. Si eran 4, se apagaba. En caso de que pasaran más de 3 segundos sin alcanzar una cantidad válida o se detectaran más de 4 aplausos, el contador se reiniciaba automáticamente

3.5. Etapa de potencia

La etapa final fue el disparo del foco de 110V con un TRIAC 2N6071. Para aislar el Arduino del voltaje de la red, se utilizó un MOC3022 como optoacoplador. El Arduino mandaba la señal al LED interno del MOC3022, y cuando se activaba, el TRIAC se disparaba, permitiendo el paso de corriente alterna al foco.

Todo esto se integró en una sola tarjeta perforada con los tres TL081, el MOC3022, el TRIAC y los componentes pasivos, dejando el Arduino y la fuente conectados por cables.

4. Código

Este fue el código utilizado explicado anteriormente:

Listing 1: Código de control PID para el carrito

```
const int pinEntrada = 2;     // Entrada del comparador
const int pinLED = 4;         // LED (simula el foco)

int contadorAplausos = 0;
unsigned long tiempoInicio = 0;
bool contando = false;
bool focoEncendido = true;

void setup() {
  pinMode(pinEntrada, INPUT);
  pinMode(pinLED, OUTPUT);
  digitalWrite(pinLED, HIGH);   // El foco inicia encendido
  Serial.begin(115200);
  Serial.println("Contador de aplausos iniciado...");
}

void loop() {
  static bool estadoAnterior = LOW;
  bool estadoActual = digitalRead(pinEntrada);

  // Detectar flanco de subida
  if (estadoActual == HIGH && estadoAnterior == LOW) {
    if (!contando) {
      tiempoInicio = millis();
      contando = true;
    }

    contadorAplausos++;
    Serial.print(" Aplauso  detectado! Total: ");
    Serial.println(contadorAplausos);

    // Acciones inmediatas
```

```
    if (contadorAplausos == 2 && !focoEncendido) {
      digitalWrite(pinLED, HIGH);
      focoEncendido = true;
      Serial.println(">> FOCO ENCENDIDO <<");
    }
    else if (contadorAplausos == 4 && focoEncendido) {
      digitalWrite(pinLED, LOW);
      focoEncendido = false;
      Serial.println(">> FOCO APAGADO <<");
    }
    else if (contadorAplausos > 4) {
      Serial.println(">> M s de 4 aplausos. Reiniciando contador. <<");
      contadorAplausos = 0;
      contando = false;
    }

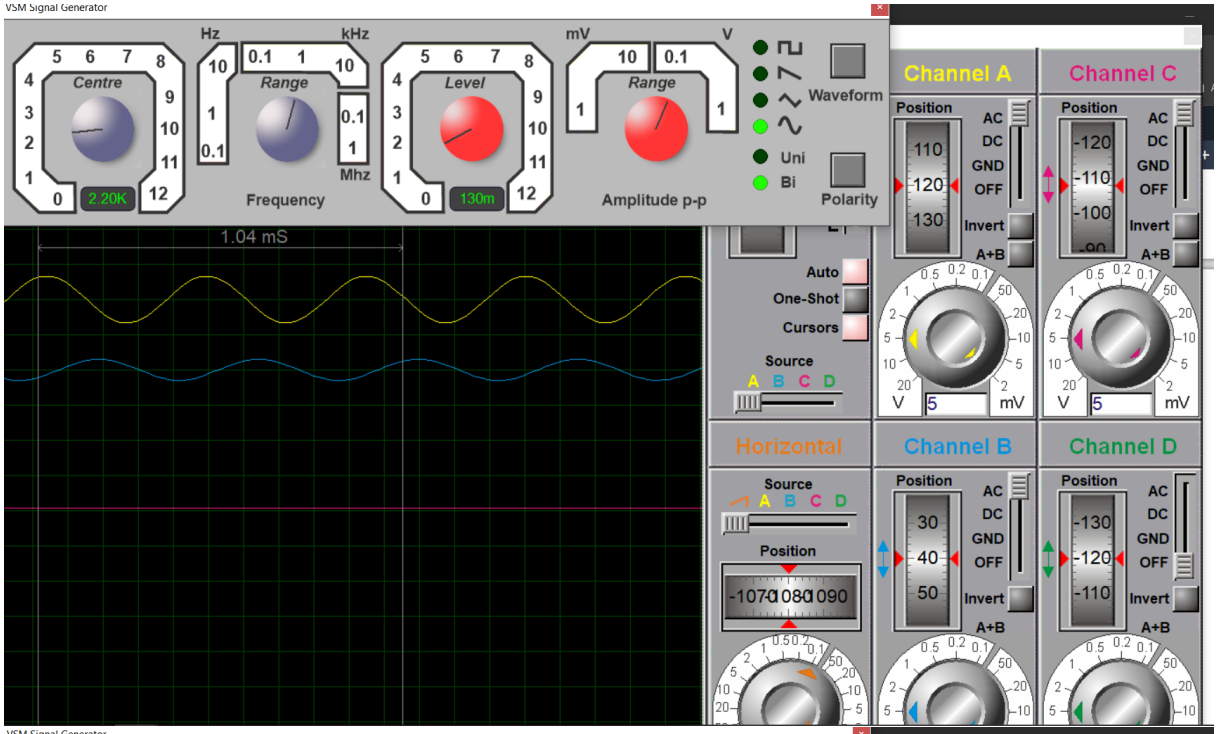
    delay(200); // Anti-rebote
  }

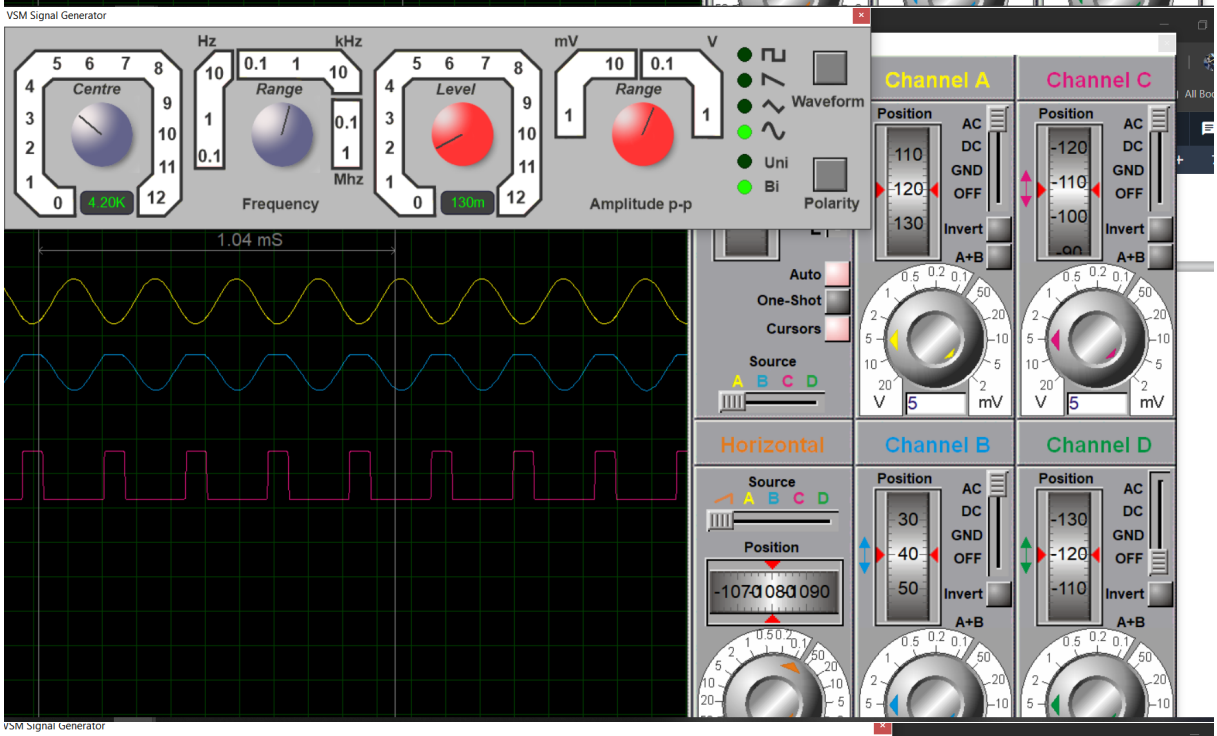
  estadoAnterior = estadoActual;

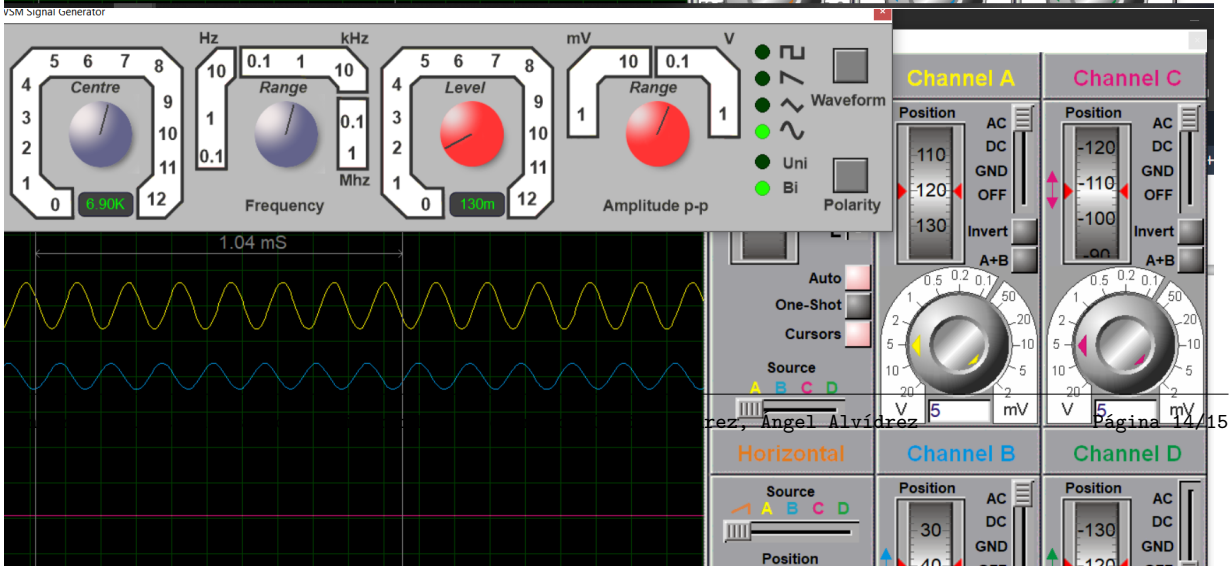
  // Si se pasa el tiempo de espera
  if (contando && (millis() - tiempoInicio >= 3000)) {
    Serial.print(">> Tiempo agotado. Aplausos contados: ");
    Serial.print(contadorAplausos);
    Serial.println(". Reiniciando contador. <<");
    contadorAplausos = 0;
    contando = false;
  }
}
```

5. Resultados

El circuito fue simulado inicialmente en Proteus, sustituyendo el micrófono por una generadora de señales. En esta simulación el comportamiento fue correcto: el filtro permitía el paso de la señal en el rango deseado y atenuaba frecuencias fuera de banda tal como se esperaba. Sin embargo, en la implementación física surgieron varios detalles que obligaron a modificar el diseño. El módulo KY-037 utilizado como micrófono introducía un offset de aproximadamente 2.5V en la señal, lo cual afectaba directamente el funcionamiento del filtro. Para mitigar este efecto, se incorporó un capacitor cerámico en la entrada del circuito con el fin de bloquear ese componente de DC, ajuste que también se aplicó posteriormente en la simulación de Proteus para reflejar el comportamiento real.


VSM Signal Generator
Centre
Range
Level
Range
Frequency
Amplitude p-p
Waveform
Polarity
Uni
Bi
Hz
kHz
mV
V
Mhz
2.20K
130m
1.04 mS
Auto
One-Shot
Cursors
Source
Channel A
Channel B
Channel C
Channel D
Horizontal
Position
AC
DC
GND
OFF
Invert
A+B


VSM Signal Generator
Centre
Range
Level
Range
Frequency
Amplitude p-p
Waveform
Polarity
Uni
Bi
Hz
kHz
mV
V
Mhz
4.20K
130m
1.04 mS
Auto
One-Shot
Cursors
Source
Channel A
Channel B
Channel C
Channel D
Horizontal
Position
AC
DC
GND
OFF
Invert
A+B


VSM Signal Generator
Centre
Range
Level
Range
Frequency
Amplitude p-p
Waveform
Polarity
Uni
Bi
Hz
kHz
mV
V
Mhz
6.90K
130m
1.04 mS
Auto
One-Shot
Cursors
Source
Channel A
Channel B
Channel C
Channel D
Horizontal
Position
AC
DC
GND
OFF
Invert
A+B

jjjjjjjujjjjjjjjjjjjjjjAdemás, se detectó que el comparador, en su estado de salida baja, introducía un pequeño voltaje negativo que resultaba no deseado al momento de excitar el gate del TRIAC. Este problema se resolvió colocando un diodo que bloquea ese valor negativo y asegura que la señal de control permanezca limpia y sin interferencias en estado de reposo. Con estos ajustes, tanto el funcionamiento en simulación como en la práctica fue estable y el circuito cumplió con su objetivo de detectar señales acústicas dentro del rango específico de frecuencias.

6. Conclusión

Este proyecto permitió integrar múltiples conceptos de electrónica analógica y digital para desarrollar un sistema funcional de detección de frecuencias específicas usando un filtro pasabanda activo con amplificadores operacionales TL081 y un circuito de disparo con TRIAC controlado por Arduino. A lo largo del desarrollo, se comprobó la importancia de considerar no solo el diseño teórico, sino también los detalles prácticos que surgen al implementar componentes reales, como el offset inesperado del módulo KY-037 o el comportamiento del comparador en estado bajo.

La simulación en Proteus fue una herramienta clave para validar el diseño antes del armado físico, pero también se evidenció que ciertos elementos no se comportan de forma idéntica entre la simulación y la práctica. Esto obligó a realizar ajustes directos sobre el circuito, como la inclusión de un capacitor para eliminar el offset de DC o un diodo para asegurar niveles de señal compatibles con el disparo del TRIAC. Estas soluciones simples pero efectivas muestran cómo una buena comprensión del comportamiento de cada bloque funcional permite resolver problemas sin rehacer todo el diseño.

Finalmente, se logró un sistema que detecta señales en el rango deseado y responde adecuadamente encendiendo una carga de corriente alterna, demostrando el uso práctico de filtros activos, comparadores y dispositivos de potencia en una aplicación de control automatizado.